

Week 5 — Tree-Based Models & Ensembles

Resource Guide & Learning Toolkit

Statistical Machine Learning · Fusemachines AI Fellowship

Facilitator: Susan Ghimire | Dataset: Telco Customer Churn

How to use this guide

This document is organised in six sections that mirror the six learning phases of the module. Work through each phase in order. **Step 1 — Understand the topic** by watching the linked YouTube videos (they are short and concept-focused). **Step 2 — Go deeper** with the textbook chapters or documentation pages listed. **Step 3 — Practise with AI prompts**: paste each prompt into ChatGPT or Claude, then use the critique guidance to evaluate and challenge the response. **Step 4 — Discuss**: bring the discussion questions to your mid-week sync. All links in this document are clickable — click any URL to open it directly. Resources marked ★ **Priority** are the minimum required before the session.

LEARNING PHASES

What you will build across six progressive phases

PHASE 1

Base Learners & Core Theory

Start with a single decision tree on raw data. Understand how the tree chooses splits (Gini Impurity, Information Gain, Entropy), trace decision boundaries, and intentionally overfit to experience the bias-variance tradeoff first-hand.

PHASE 2

Data Prep & Evaluation

Fix the naïve model. Handle class imbalance and missing values. Learn why accuracy is a dangerous metric on imbalanced data (the Accuracy Trap) and master AUROC, Precision, Recall, and F1-Score.

PHASE 3

Building the Crowd — Ensembles

Combine many trees into a powerful ensemble. Understand how Bagging reduces variance through parallel diversity, and how Random Forests add feature randomness on top of bagging to decorrelate the individual trees.

PHASE 4

State-of-the-Art & Optimisation

Move from parallel to sequential ensembling. Study XGBoost, LightGBM, and CatBoost — each adding trees to fix the residual errors of the previous round. Tune hyperparameters systematically with Grid Search and Bayesian optimisation.

PHASE 5

Workflow Management & Interpretability

Wrap everything in a Scikit-Learn Pipeline to eliminate data leakage. Then open the black box: use SHAP values to explain individual predictions and global feature importance to business stakeholders.

PHASE 6

Deployment Prep

Save your fitted pipeline — not just the model — using joblib with semantic versioning. Write a model card covering metrics, thresholds, and known limitations.

RESOURCES

Curated materials — click any link to open directly

Reading strategy: If you have 1–2 hours, watch the YouTube videos first — they give you the mental model. If you have 4+ hours, pair each video with its corresponding textbook chapter. For production code, always cross-check the official scikit-learn and SHAP documentation. The XGBoost paper is optional but strongly recommended if you want to understand regularisation from first principles.

★ Priority Videos — Watch Before the Session

■ Decision and Classification Trees, Clearly Explained!!!

<https://www.youtube.com/watch?v=L39rN6qz7Y&t=18s>

★ Priority · Phase 1. StatQuest's flagship decision tree video. Covers Gini impurity and the full tree-building algorithm with animated examples. **Watch this first.**

■ Entropy (for data science), Clearly Explained!!!

<https://youtu.be/YtebGVx-Fxw>

★ Priority · Phase 1. Understand information entropy and how it is used as an alternative splitting criterion. Pairs directly with the Gini video above.

■ ROC and AUC, Clearly Explained!

<https://youtu.be/4jRBRDbJemM>

★ Priority · Phase 2. Essential for understanding AUROC — the primary metric used throughout the assignment. Explains the confusion matrix, thresholds, and the diagonal baseline.

■ StatQuest: Random Forests Part 1 — Building, Using and Evaluating

https://youtu.be/J4Wdy0Wc_xQ

★ Priority · Phase 3. Builds on the decision tree video. Explains bootstrap sampling, OOB error, and why feature randomness reduces correlation between trees.

■ XGBoost Part 1 of 4 — Regression

<https://youtu.be/OtD8wVaFm6E>

★ Priority · Phase 4. Josh Starmer explains gradient boosting from scratch. Covers residuals, learning rate, and the sequential correction loop.

■ Shapley Additive Explanations (SHAP)

<https://youtu.be/VB9uV-x0gtg>

Phase 5. Conceptual introduction to SHAP using game-theory intuition. Watch before generating SHAP plots in the notebook.

■ Lecture 9 — Decision Trees & Ensemble Methods · Stanford CS229

<https://youtu.be/wr9gUr-eWdA>

Optional — deeper theory. Full lecture covering trees, bagging, boosting, and random forests with mathematical notation.

Textbooks & Long-Form Reading

■ Hands-On ML with Scikit-Learn, Keras & TensorFlow (Géron) — Ch. 6 & 7

<https://github.com/ageron/handson-ml3>

★ Priority if you have time · Phases 1–3. Chapter 6 covers decision trees with code. Chapter 7 covers all ensemble methods. Both chapters include runnable Colab notebooks. Start with Chapter 6 alongside Phase 1.

■ An Introduction to Statistical Learning — Ch. 8: Tree-Based Methods

<https://www.statlearning.com/>

★ Priority if you want theory · Phases 1–3. Free PDF. The most accessible rigorous treatment of trees, bagging, boosting, and random forests. Chapter 8 is 50 pages and self-contained. Recommended over ESL for most students.

■ The Elements of Statistical Learning (Hastie, Tibshirani, Friedman)

<https://hastie.su.domains/ElemStatLearn/>

Optional — advanced. The mathematical foundation behind gradient boosting. Read Ch. 10 (Boosting) if you want to understand AdaBoost and gradient boosting from first principles.

■ Interpretable Machine Learning (Molnar) — SHAP Chapter

<https://christophm.github.io/interpretable-ml-book/shap.html>

Phase 5. Free online book. The SHAP chapter explains Shapley values, SHAP summary plots, and force plots in plain language. Read alongside the SHAP video.

Official Documentation & Reference

■ Scikit-Learn — Decision Trees

<https://scikit-learn.org/stable/modules/tree.html>

Phase 1. Official API reference and user guide for DecisionTreeClassifier and DecisionTreeRegressor. Includes the parameters that prevent overfitting (max_depth, min_samples_split, min_samples_leaf).

■ Scikit-Learn — Ensemble Methods

<https://scikit-learn.org/stable/modules/ensemble.html>

Phases 3–4. Covers BaggingClassifier, RandomForestClassifier, GradientBoostingClassifier, and HistGradientBoosting. Read alongside the StatQuest videos.

■ Scikit-Learn — Common Pitfalls & Recommended Practices

https://scikit-learn.org/stable/common_pitfalls.html

Phase 5 — Essential. Short, practical page on data leakage, pipeline design, and cross-validation mistakes. Read before building your pipeline.

■ XGBoost — Introduction to Boosted Trees

<https://xgboost.readthedocs.io/en/stable/tutorials/model.html>

Phase 4. The official XGBoost tutorial explaining the objective function, regularisation (reg_alpha, reg_lambda), and the tree construction algorithm.

■ SHAP Documentation

<https://shap.readthedocs.io/en/latest/>

Phase 5. API reference for shap.TreeExplainer, summary_plot, force_plot, and waterfall_plot. Use this when generating SHAP visualisations in the notebook.

■ Kaggle — When to Use XGBoost | LightGBM | CatBoost

<https://www.kaggle.com/code/masayakawamata/when-to-use-xgboost-lightgbm-catboost>

Phase 4. Practical side-by-side comparison notebook. Useful for deciding which framework to use and understanding CatBoost's built-in categorical handling.

■ Kaggle — Hyperparameter Tuning in Random Forests

<https://www.kaggle.com/code/nargisbegum82/hyperparameter-tuning-in-random-forests>

Phase 4. Worked example of GridSearchCV with RandomForestClassifier. Good template for your own tuning code.

Dataset

■ Telco Customer Churn Dataset · Kaggle

<https://www.kaggle.com/datasets/blatchar/telco-customer-churn/data>

7,043 rows · 21 columns. The single dataset used across all six phases and both the classification and regression parts of the assignment. Known issues you must handle: TotalCharges contains whitespace-only strings that read as object dtype (coerce to numeric), ~27% churn rate (class imbalance), and mixed numeric / categorical feature types.

AI PROMPTS & CRITIQUE GUIDE

How to use a language model as a learning tool — not a shortcut

How to use these prompts: Paste each prompt into [Claude](#) or [ChatGPT](#). Read the response carefully, then apply the **Critique / Verify** guidance below it to actively evaluate whether the answer is correct, incomplete, or misleading. The goal is **critical engagement** — you are the expert checking the AI's work, not a passive reader. If the model gets something wrong, ask a follow-up that forces it to fix the mistake. Write a short note in your notebook about what it got right and what it missed.

Phase 1 — Decision Trees

Ask: "Write a Scikit-Learn pipeline that uses SMOTE to balance data and trains a RandomForestClassifier using cross-validation."

Critique / Verify: Did the model use `sklearn.pipeline.Pipeline` (wrong — causes leakage) or `imblearn.pipeline.Pipeline` (correct)? If it used sklearn's Pipeline, explain in your own words exactly why SMOTE inside a standard pipeline causes data leakage. Force the model to correct itself.

Ask: "Explain step-by-step how a decision tree chooses the best feature and threshold to split on at each node using Gini Impurity."

Critique / Verify: Check: does it give the correct formula $Gini = 1 - \sum p_i^2$? Does it explain that the tree evaluates every possible threshold for every feature and picks the one with the lowest weighted child impurity? Ask it to show a worked numerical example.

Phase 2 — Evaluation

Ask: "My decision tree achieves 85% accuracy on a dataset where 85% of samples are class 0. Is this a good result? Why or why not?"

Critique / Verify: The model should identify this as the **Accuracy Trap** and explain that a dummy classifier predicting class 0 every time achieves the same score. It should recommend Recall, Precision, F1, and AUROC. If it just says 'yes that is good', it is wrong.

Ask: "Explain the difference between Precision and Recall and when you would prioritise each metric in a customer churn prediction problem."

Critique / Verify: A correct answer: high Recall = catch as many churners as possible (cost of missing a churner is high); high Precision = only flag customers you are confident will churn (cost of wasting a retention call is high). Ask the model to calculate Precision and Recall from a 2x2 confusion matrix to verify it knows the formulas.

Phase 3 — Ensembles

Ask: "Explain the mathematical difference between how a Random Forest reduces variance and how XGBoost reduces bias."

Critique / Verify: Random Forest: independent trees trained on bootstrap samples with random feature subsets → averaging reduces variance. XGBoost: sequential trees each correcting the residuals of the previous → reduces bias. The model should clearly articulate this distinction. Push back if it conflates the two.

Ask: "What is an Out-Of-Bag (OOB) score and why does it give a reliable estimate of generalisation error without a separate validation set?"

Critique / Verify: Each bootstrap sample omits ~37% of training points. Those omitted samples are OOB for that tree and can be used as a validation set. The model should explain this and note that OOB error is effectively free cross-validation.

Phase 4 — Boosting & Tuning

Ask: "What does `reg_alpha` do in XGBoost, and how does it differ from `reg_lambda`?"

Critique / Verify: `reg_alpha` = L1 (Lasso) regularisation on leaf weights. `reg_lambda` = L2 (Ridge). L1 can shrink weights to exactly zero (sparse trees); L2 penalises large weights without zeroing. If the model says `reg_alpha` is the learning rate it is completely wrong.

Ask: "How does a Decision Tree Regressor make predictions for data points that are mathematically outside the range of its training data?"

Critique / Verify: The model must admit that trees **cannot extrapolate**. Predictions are capped at the mean of the nearest training leaf — if your training data goes up to 72 months, no prediction will exceed 72. This is a structural limitation. If the AI suggests the tree interpolates or extrapolates, it is wrong.

Phase 5 — Pipelines & SHAP

Ask: "Explain how SHAP values distribute the 'payout' of a prediction among features using Game Theory."

Critique / Verify: SHAP values are Shapley values from cooperative game theory. Each feature is a 'player', and the prediction is the 'payout'. The Shapley value for a feature is the average marginal contribution across all possible orderings of features. Ask the model whether it can make this understandable for a non-technical manager.

MID-WEEK DISCUSSION QUESTIONS

Designed to surface misconceptions — bring your reasoning, not just your answer

How to use these questions: These are deliberately open-ended and adversarial — they are designed so that a superficial answer reveals a gap. Before the discussion, write a 2–3 sentence answer in your notebook. During the session, compare your reasoning with peers. There is rarely a single right answer; the quality of your justification matters more than the conclusion.

Q1. Accuracy Trap in Production

You built an XGBoost model for churn with 99% accuracy on the test set, but it has no constraints (`max_depth=None`). What is about to happen in production, and how would you have caught this during development?

Q2. SMOTE Leakage

Your pipeline applies SMOTE to X and y *before* running `cross_val_score`. Your AUROC is 0.95. Why should you be concerned, and what does the true AUROC likely look like?

Q3. Model Selection Trade-off

A Random Forest and an XGBoost model both achieve 0.85 F1-Score. The Random Forest trains in 2 seconds; the tuned XGBoost takes 20 minutes. Which do you deploy and why? What other factors would change your answer?

Q4. SHAP-Driven Business Decision

The SHAP force plot for a churned customer shows `Tenure = 1 month` pushing the prediction higher, but `InternetService = Fiber` pushing it lower. The retention agent asks what discount to offer. What do you tell them — and what should you NOT say?

Q5. Bayesian vs Grid Search

You tuned your LightGBM model using Grid Search over 5 parameters and it took 4 hours. A colleague suggests Bayesian Search. Why would that be faster, and what is the underlying logic of how it navigates the hyperparameter space?

Q6. Tree Regressor Extrapolation

Your Random Forest regression model for CLV completely fails to predict values higher than the maximum in the training data. Why is this structurally inevitable for tree-based regressors, and how does it differ from last week's Linear Regression?

Q7. CatBoost vs XGBoost Encoding

You used CatBoost and XGBoost on the same dataset. CatBoost did not require One-Hot Encoding but XGBoost did. Under the hood, how does CatBoost handle text categories without creating target leakage?

Q8. Feature Importance vs SHAP Disagreement

Your tree's native `feature_importances_` array ranks `ContractType` as most important. The SHAP summary plot ranks `MonthlyCharges` as most important. Why might these two explainers disagree, and which one would you trust for a business stakeholder report?

At a glance: which resource to use for each phase

Phase	Topic	★ Watch	★ Read / Docs
1	Decision Trees & Splitting	Trees Clearly Explained Entropy Clearly Explained	Géron Ch. 6 ISLR Ch. 8
2	Evaluation & Imbalanced Data	ROC & AUC Clearly Explained	sklearn Common Pitfalls
3	Ensembles & Random Forest	StatQuest: Random Forests	Géron Ch. 7 ISLR Ch. 8
4	Boosting & Hyperparameter Tuning	XGBoost Part 1	XGBoost Docs: Boosted Trees Kaggle: XGB/LGBM/CAT
5	Pipelines & SHAP	SHAP Explained	Molnar: SHAP Chapter SHAP Docs
6	Serialisation & Model Cards	—	sklearn Pipeline Docs